01_CODE_ALONG_LAB

Pintop | 2026 Training

CLASSROOM CODE-ALONG - REST Principles & Resource Modelling (Backend Design & Engineering)

Pintop Technologies | Class 01 | Instructor and student companion

What we will produce

Recreate a documented excerpt from the **real project** (`routes/api.php`), practise a small safe variation, verify using the complete app or appropriate tool, and record both success and failure evidence. Preserve the original project.

Why: A mobile app team asks you for 'the order API'. If you answer with a jumble of random URLs, they will waste days guessing. Today we learn to model the domain as resources with clear names, so the API explains itself.

1. Find and read the real file

`codebase/routes/api.php`

The excerpt is in `REFERENCE EXCERPT.php`. Its job is to give you something to type, trace and understand. It is not guaranteed to run on its own: it is part of a larger file. Do not overwrite project source with a fragment.

2. Set up the real project

```
cd "Backend Design & Engineering/codebase"
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve --port=8000
```

3. Type in stages

Stage 1

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
use App\Http\Controllers\PayTrackController;
use App\Http\Controllers\PayTrackAuthController;
Route::get('/health/live',fn()=>response()->json(['status'=>'alive']));
Route::get('/health/ready',[PayTrackController::class,'health']);
Route::prefix('v1')->group(function(){
 Route::post('/auth/register',[PayTrackController::class,'register']);
```

Stage 2

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
Route::post('/customers',[PayTrackController::class,'createCustomer']);
Route::post('/auth/login',[PayTrackAuthController::class,'login']);
Route::post('/auth/refresh',[PayTrackAuthController::class,'refresh']);
Route::middleware('training.token')->group(function(){
 Route::get('/customers/{id}',[PayTrackController::class,'customer']);
 Route::get('/wallets/{id}',[PayTrackController::class,'wallet']);
```

Stage 3

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
 Route::get('/wallets/{id}/transactions',[PayTrackController::class,'ledger']);
 Route::post('/transfers',[PayTrackController::class,'transfer']);
 Route::post('/admin/refunds',[PayTrackController::class,'refunds']);
 });
});
```

4. Complete these activities

1. **Open the manifest or workflow and trace one setting from configuration to runtime.**
2. **Change a safe non-secret value in a disposable local copy; validate the file before applying it.**
3. **Show the verification command and document how to undo the change.**

5. Check, debug and record

```
php artisan route:list --path=api
php artisan test
```

Expected check: If the local app starts or its tests pass, capture evidence of the specific behaviour tied to this class, not merely an application startup. Test one failed or edge input too. Log the exact error if blocked.

6. Marking

- 4/20: explain concept and input/output in plain language
- 4/20: accurate typed or edited work
- 4/20: local proof and explanation
- 4/20: safe variation and wrong-case recovery
- 4/20: short documented submission

Lab ethics: no unapproved public scanning, no third-party tokens and no real banking customer data.